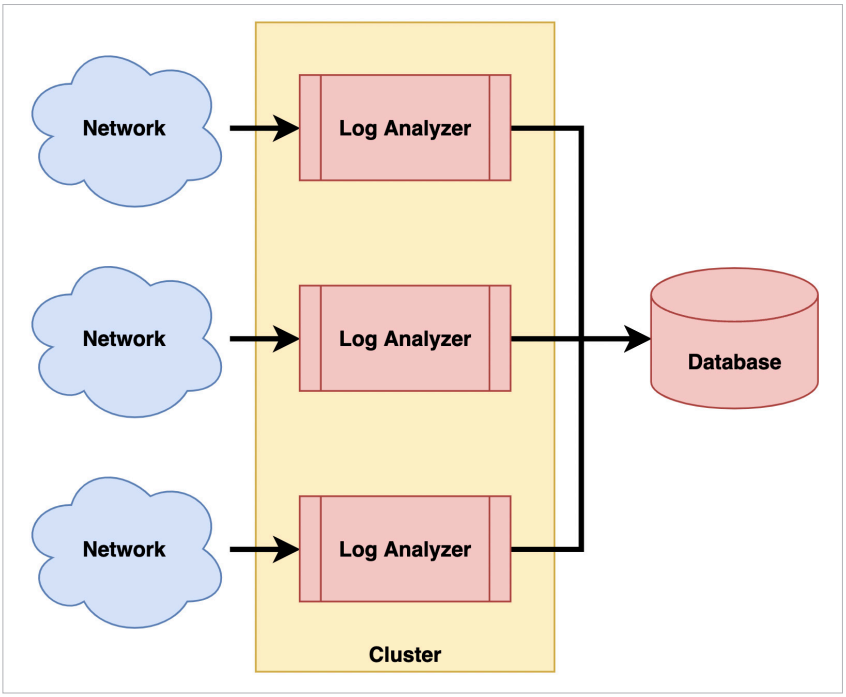

Figure 4: Distributed log analyser

to handle a minimum of 100 logs per second. Our product marketing team determined through their research that this level of performance would effectively accommodate the size of the networks.

I began my work by exploring avenues for performance enhancement. I fine-tuned database operations, string processing routines, and other aspects, hoping to save a few CPU cycles. Unfortunately, this approach did not yield much success.

With no room for further enhancement within the current framework, I opted to revamp the system using a hub-and-spoke architecture, depicted in Figure 3.

Unlike the previous setup where a single thread processed logs sequentially, the redesigned system featured multiple threads: collectors, analysers, and processors.

The collector thread retrieves logs from the UDP port as they become available and promptly deposits them into the analyser queue. This thread solely focuses on log collection, ensuring that all logs are captured by the system without any being dropped.

The analyser thread retrieves logs from the analyser queue, standardises them, and applies filters. Logs that meet the filter criteria are then transferred to the processor queue. When operating on multi-core processors, multiple analyser threads can be scheduled simultaneously to enhance performance.

Subsequently, processor threads retrieve logs from the processor queue. Various types of processor threads can be implemented. For instance, one thread may store logs in the database, another may send emails, and yet another may trigger page alerts based on configured actions.

Although the processing speed of the analyser and processor threads may be slightly slower, this does not pose a problem as the collector thread continuously writes new logs to the collector queue. By adjusting the queue sizes and the number of threads, the system can dynamically scale to handle higher loads.

However, does this design hold up well now? Certainly not! With the proliferation of intelligent devices, IoT devices, and so forth, network sizes have grown exponentially. While the multi-threaded design illustrated in Figure 3 can scale to some extent, it has its limitations in vertical scaling.

For contemporary requirements, I would opt for horizontal scaling over vertical scaling. This means redesigning the system as a distributed system, as depicted in Figure 4.

In this approach, the log analyser is replicated to form a cluster. The size of the cluster can be adjusted based on demand. Each instance of the log analyser handles logs from only a few devices, ensuring that the load on each log analyser remains manageable. Network administrators configure which devices send logs to which instances.

Additionally, log analysers can be deployed on the cloud with appropriate elasticity rules. This allows optimal hardware utilisation while smoothly scaling the cluster up or down to handle varying loads.

Looking at the current landscape, there are numerous log processing systems available in the market which follow such distributed design. Instead of building yet another log processing system, one could opt to acquire machine data analysers like Splunk for their needs.

Technology continues to evolve rapidly. END

By: Krishna Mohan Koyya

The author is the CEO and managing director of Koyya Enterprises Private Limited in Bengaluru. With over twenty years of experience, he has held various roles in the technology sector.